

1. Conservative smoothing filter

```
function filtered_image = conservative_smoothing(image)

[rows, cols, channels] = size(image);
filtered_image = image;

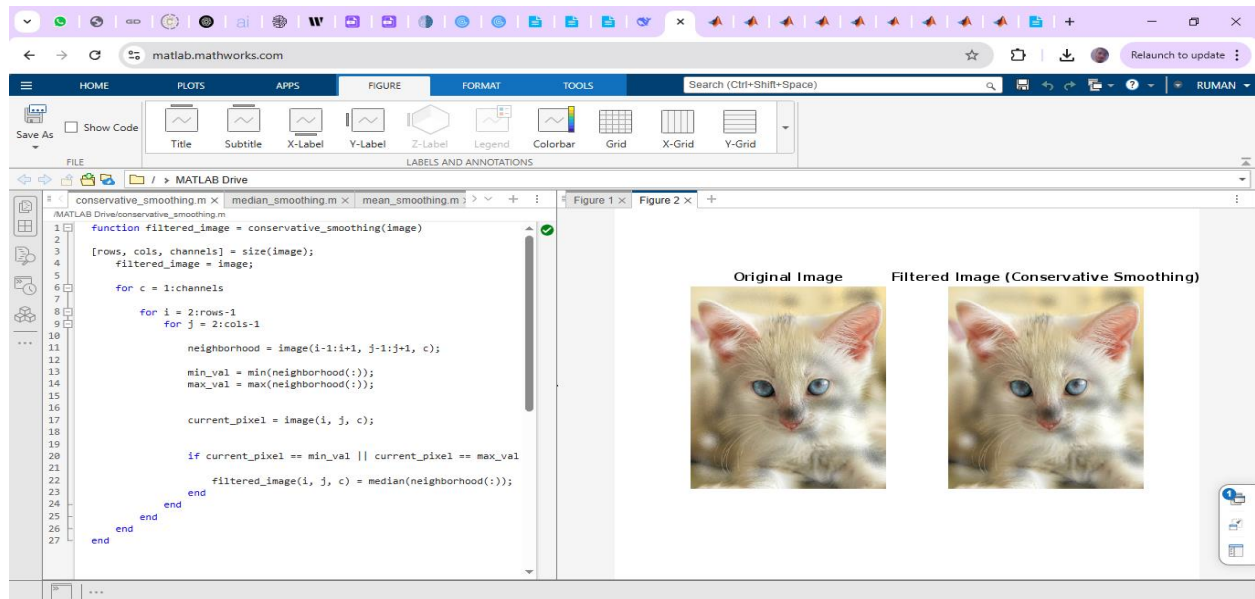
for c = 1:channels
    for i = 2:rows-1
        for j = 2:cols-1

            neighborhood = image(i-1:i+1, j-1:j+1, c);

            min_val = min(neighborhood(:));
            max_val = max(neighborhood(:));

            current_pixel = image(i, j, c);

            if current_pixel == min_val || current_pixel == max_val
                filtered_image(i, j, c) = median(neighborhood(:));
            end
        end
    end
end
```



2. Median smoothing filter

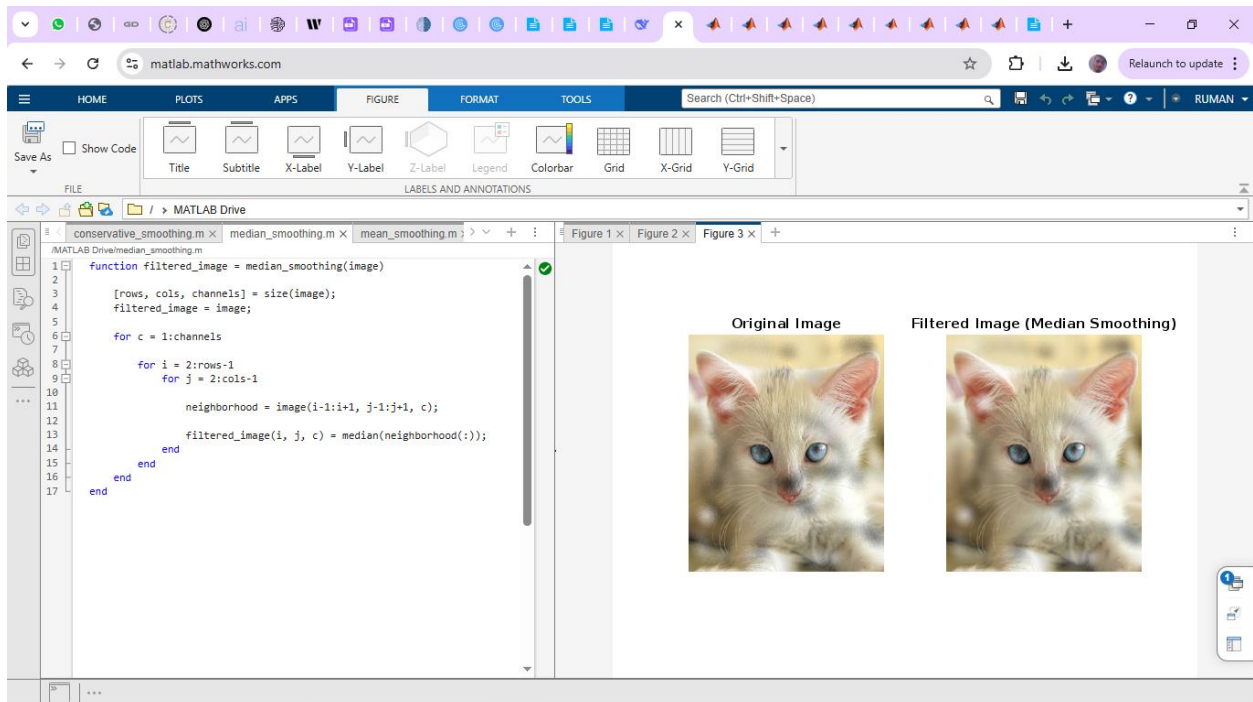
```
function filtered_image = median_smoothing(image)

[rows, cols, channels] = size(image);
filtered_image = image;

for c = 1:channels
    for i = 2:rows-1
        for j = 2:cols-1

            neighborhood = image(i-1:i+1, j-1:j+1, c);

            filtered_image(i, j, c) = median(neighborhood(:));
        end
    end
end
```



3. Mean smoothing filter

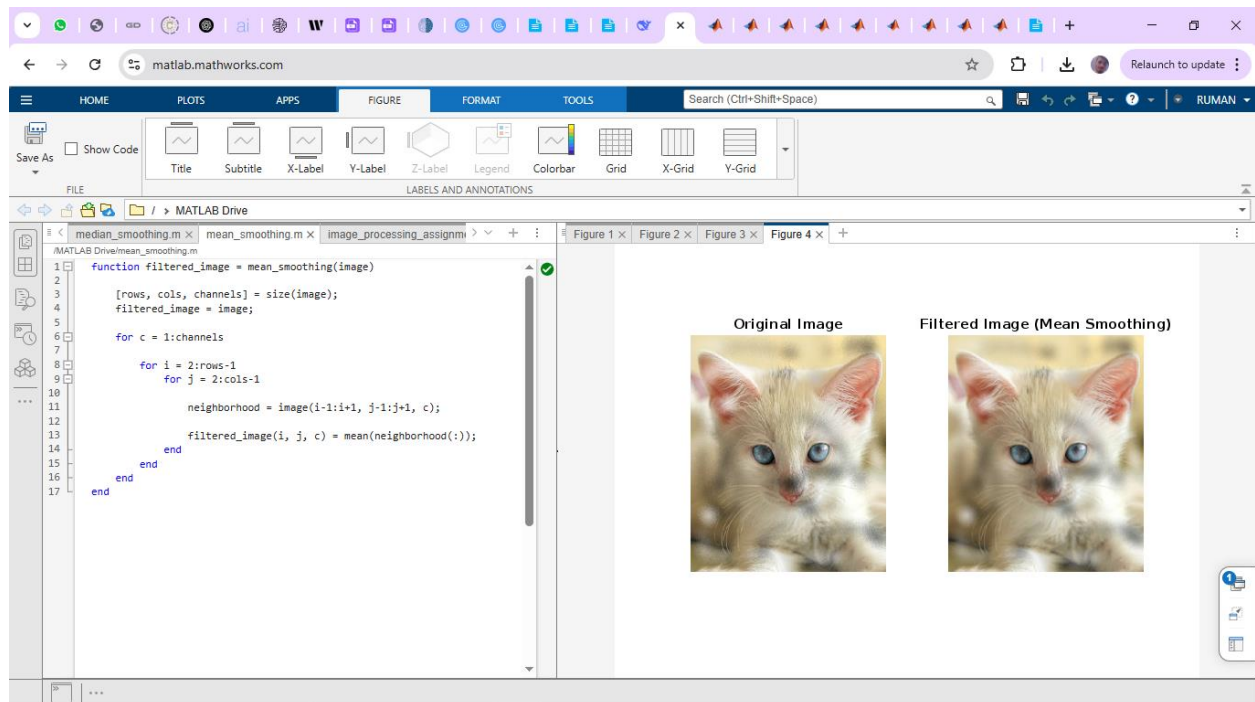
```
function filtered_image = mean_smoothing(image)

[rows, cols, channels] = size(image);
filtered_image = image;

for c = 1:channels
    for i = 2:rows-1
        for j = 2:cols-1

            neighborhood = image(i-1:i+1, j-1:j+1, c);

            filtered_image(i, j, c) = mean(neighborhood(:));
        end
    end
end
```



4. Applying 3 smoothing filters

```
image = imread('noisy_image.jpg');

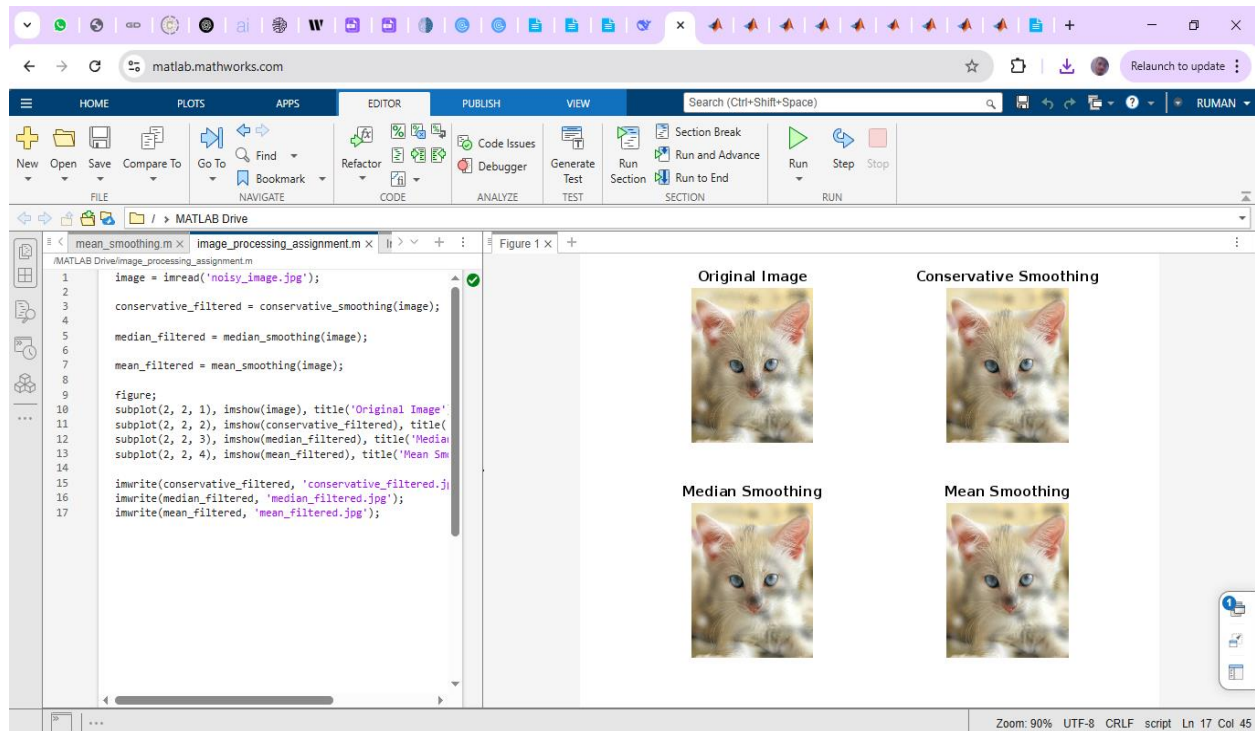
conservative_filtered = conservative_smoothing(image);

median_filtered = median_smoothing(image);

mean_filtered = mean_smoothing(image);

figure;
subplot(2, 2, 1), imshow(image), title('Original Image');
subplot(2, 2, 2), imshow(conservative_filtered), title('Conservative Smoothing');
subplot(2, 2, 3), imshow(median_filtered), title('Median Smoothing');
subplot(2, 2, 4), imshow(mean_filtered), title('Mean Smoothing');

imwrite(conservative_filtered, 'conservative_filtered.jpg');
imwrite(median_filtered, 'median_filtered.jpg');
imwrite(mean_filtered, 'mean_filtered.jpg');
```



5. GUI development using App designer

```

classdef ImageProcessingApp < matlab.apps.AppBase

    properties (Access = public)
        UIFigure          matlab.ui.Figure
        OriginalImageAxes  matlab.ui.control.UIAxes
        FilteredImageAxes  matlab.ui.control.UIAxes
        OpenImageButton    matlab.ui.control.Button
        ConservativeButton matlab.ui.control.Button
        MedianButton        matlab.ui.control.Button
        MeanButton          matlab.ui.control.Button
        SaveButton          matlab.ui.control.Button
    end

    properties (Access = private)
        OriginalImage
        FilteredImage
    end

    methods (Access = private)

        function OpenImageButtonPushed(app, ~)

            [filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp', 'Image Files (*.jpg, *.png, *.bmp)'});
            if isequal(filename, 0)
                return;
            end
            app.OriginalImage = imread(fullfile(pathname, filename));

            imshow(app.OriginalImage, 'Parent', app.OriginalImageAxes);
        end

        function ConservativeButtonPushed(app, ~)

            if ~isempty(app.OriginalImage)
                app.FilteredImage = conservative_smoothing(app.OriginalImage);
                imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
            end
        end

        function MedianButtonPushed(app, ~)

            if ~isempty(app.OriginalImage)
                app.FilteredImage = median_smoothing(app.OriginalImage);
                imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
            end
        end

        function MeanButtonPushed(app, ~)

            if ~isempty(app.OriginalImage)

```

```

        app.FilteredImage = mean_smoothing(app.OriginalImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
    end
end

function SaveButtonPushed(app, ~)

    if ~isempty(app.FilteredImage)
        [filename, pathname] = uiputfile({'*.jpg', 'JPEG Image (*.jpg)'});
        if isequal(filename, 0)
            return;
        end
        imwrite(app.FilteredImage, fullfile(pathname, filename));
    end
end

methods (Access = private)

    function createComponents(app)

        app.UIFigure = uifigure;
        app.UIFigure.Position = [100 100 800 600];
        app.UIFigure.Name = 'Image Processing App';

        app.OriginalImageAxes = uiaxes(app.UIFigure);
        app.OriginalImageAxes.Position = [50 350 300 200];
        app.OriginalImageAxes.XTick = [];
        app.OriginalImageAxes.YTick = [];
        title(app.OriginalImageAxes, 'Original Image');

        app.FilteredImageAxes = uiaxes(app.UIFigure);
        app.FilteredImageAxes.Position = [450 350 300 200];
        app.FilteredImageAxes.XTick = [];
        app.FilteredImageAxes.YTick = [];
        title(app.FilteredImageAxes, 'Filtered Image');

        app.OpenImageButton = uibutton(app.UIFigure, 'push');
        app.OpenImageButton.ButtonPushedFcn = createCallbackFcn(app,
@OpenImageButtonPushed, true);
        app.OpenImageButton.Position = [50 300 100 30];
        app.OpenImageButton.Text = 'Open Image';

        app.ConservativeButton = uibutton(app.UIFigure, 'push');
        app.ConservativeButton.ButtonPushedFcn = createCallbackFcn(app,
@ConservativeButtonPushed, true);
        app.ConservativeButton.Position = [50 250 100 30];
        app.ConservativeButton.Text = 'Conservative';

        app.MedianButton = uibutton(app.UIFigure, 'push');
        app.MedianButton.ButtonPushedFcn = createCallbackFcn(app,
@MedianButtonPushed, true);
        app.MedianButton.Position = [50 200 100 30];
        app.MedianButton.Text = 'Median';
    end
end

```

```

app.MeanButton = uibutton(app.UIFigure, 'push');
app.MeanButton.ButtonPushedFcn = createCallbackFcn(app,
@MeanButtonPushed, true);
app.MeanButton.Position = [50 150 100 30];
app.MeanButton.Text = 'Mean';

app.SaveButton = uibutton(app.UIFigure, 'push');
app.SaveButton.ButtonPushedFcn = createCallbackFcn(app,
@SaveButtonPushed, true);
app.SaveButton.Position = [50 100 100 30];
app.SaveButton.Text = 'Save Image';

end
end

methods (Access = public)

function app = ImageProcessingApp

    createComponents(app)

    registerApp(app, app.UIFigure)

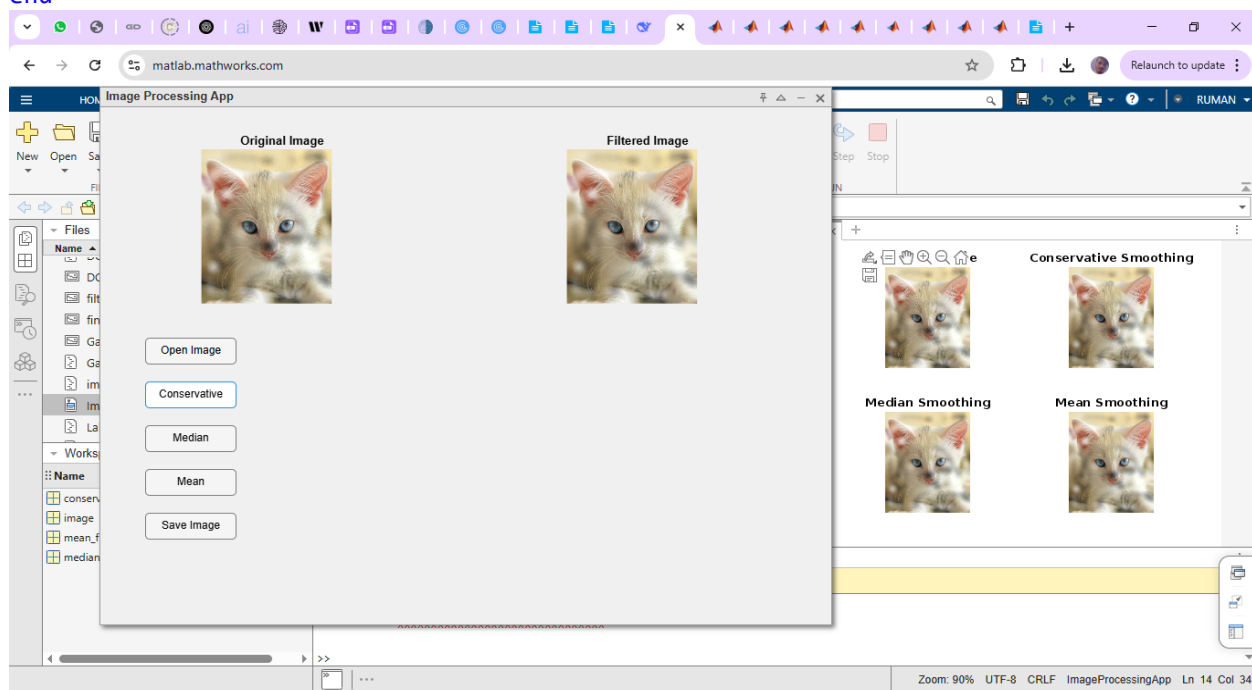
    runStartupFcn(app, @startupFcn)
end

function delete(app)

    delete(app.UIFigure)
end

end
end
end

```



6. GUI development using App designer with additional work

```

classdef ImageProcessingApp_Final < matlab.apps.AppBase

    properties (Access = public)

        UIFigure            matlab.ui.Figure
        OriginalImageAxes    matlab.ui.control.UIAxes
        FilteredImageAxes    matlab.ui.control.UIAxes
        OpenImageButton      matlab.ui.control.Button
        ConservativeButton    matlab.ui.control.Button
        MedianButton          matlab.ui.control.Button
        MeanButton            matlab.ui.control.Button
        SharpenButton         matlab.ui.control.Button
        ResetButton           matlab.ui.control.Button
        SaveButton            matlab.ui.control.Button
        BrightnessSlider      matlab.ui.control.Slider
        ContrastSlider        matlab.ui.control.Slider
        BrightnessLabel       matlab.ui.control.Label
        ContrastLabel         matlab.ui.control.Label
    end

    properties (Access = private)

        OriginalImage
        FilteredImage
        CurrentImage
    end

    methods (Access = private)

        function OpenImageButtonPushed(app, ~)
            [filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp', 'Image Files'});
            if ~isequal(filename, 0)
                app.OriginalImage = imread(fullfile(pathname, filename));
                app.CurrentImage = app.OriginalImage;
                imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
                title(app.OriginalImageAxes, 'Original Image');
                app.BrightnessSlider.Value = 0;
                app.ContrastSlider.Value = 0;
            end
        end

        function filtered = conservativeFilter(~, img)
            [rows, cols, ch] = size(img);
            filtered = img;
            for c = 1:ch
                for i = 2:rows-1
                    for j = 2:cols-1
                        neighborhood = img(i-1:i+1,j-1:j+1,c);
                        min_val = min(neighborhood(:));
                        max_val = max(neighborhood(:));
                    end
                end
            end
        end
    end
end

```



```

        if img(i,j,c) == min_val || img(i,j,c) == max_val
            filtered(i,j,c) = median(neighborhood(:));
        end
    end
end

function filtered = medianFilter(~, img)
    filtered = medfilt3(img);
end

function filtered = meanFilter(~, img)
    kernel = ones(3)/9;
    filtered = imfilter(img, kernel);
end

function ConservativeButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        app.FilteredImage = app.conservativeFilter(app.CurrentImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
        title(app.FilteredImageAxes, 'Conservative Filter');
    end
end

function MedianButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        app.FilteredImage = app.medianFilter(app.CurrentImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
        title(app.FilteredImageAxes, 'Median Filter');
    end
end

function MeanButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        app.FilteredImage = app.meanFilter(app.CurrentImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
        title(app.FilteredImageAxes, 'Mean Filter');
    end
end

function BrightnessSliderValueChanged(app, ~)
    if ~isempty(app.CurrentImage)
        brightness_value = app.BrightnessSlider.Value * 2.55;
        enhanced = double(app.CurrentImage) + brightness_value;
        app.CurrentImage = uint8(min(max(enhanced, 0), 255));
        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function ContrastSliderValueChanged(app, ~)
    if ~isempty(app.CurrentImage)
        factor = (app.ContrastSlider.Value + 100)/100;
        enhanced = (double(app.CurrentImage) - 128) * factor + 128;
        app.CurrentImage = uint8(min(max(enhanced, 0), 255));
    end
end

```

```

        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function SharpenButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        kernel = [0 -1 0; -1 5 -1; 0 -1 0];
        sharpened = imfilter(double(app.CurrentImage), kernel, 'conv');
        app.CurrentImage = uint8(min(max(sharpened, 0), 255));
        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function ResetButtonPushed(app, ~)
    if ~isempty(app.OriginalImage)
        app.CurrentImage = app.OriginalImage;
        app.BrightnessSlider.Value = 0;
        app.ContrastSlider.Value = 0;
        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function SaveButtonPushed(app, ~)
    if ~isempty(app.FilteredImage)
        [filename, pathname] = uiputfile({'*.jpg', 'JPEG Image'});
        if ~isequal(filename, 0)
            imwrite(app.FilteredImage, fullfile(pathname, filename));
        end
    end
end

end

methods (Access = private)
    function createComponents(app)

        app.UIFigure = uifigure('Name', 'Image Processing App');
        app.UIFigure.Position = [100 100 900 600];

        app.OriginalImageAxes = uiaxes(app.UIFigure);
        app.OriginalImageAxes.Position = [50 300 350 250];
        title(app.OriginalImageAxes, 'Original Image');

        app.FilteredImageAxes = uiaxes(app.UIFigure);
        app.FilteredImageAxes.Position = [500 300 350 250];
        title(app.FilteredImageAxes, 'Filtered Image');

        btnStyle = {'FontWeight', 'bold', 'FontSize', 12};

        app.OpenImageButton = uibutton(app.UIFigure, btnStyle{:}, ...
            'Position', [50 250 120 30], 'Text', 'Open Image');

        app.ConservativeButton = uibutton(app.UIFigure, btnStyle{:}, ...
            'Position', [50 210 120 30], 'Text', 'Conservative');

        app.MedianButton = uibutton(app.UIFigure, btnStyle{:}, ...

```

```

        'Position', [50 170 120 30], 'Text', 'Median');

app.MeanButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [50 130 120 30], 'Text', 'Mean');

app.SharpenButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [500 250 120 30], 'Text', 'Sharpen');

app.ResetButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [500 210 120 30], 'Text', 'Reset');

app.SaveButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [500 170 120 30], 'Text', 'Save Image');

app.BrightnessSlider = uislider(app.UIFigure, ...
    'Position', [200 250 150 20], 'Limits', [-100 100]);

app.ContrastSlider = uislider(app.UIFigure, ...
    'Position', [200 210 150 20], 'Limits', [-100 100]);

app.BrightnessLabel = uilabel(app.UIFigure, ...
    'Position', [200 270 150 20], 'Text', 'Brightness');

app.ContrastLabel = uilabel(app.UIFigure, ...
    'Position', [300 270 150 20], 'Text', 'Contrast');
    end
end

methods (Access = public)
    function app = ImageProcessingApp_Final
        createComponents(app);

        app.OpenImageButton.ButtonPushedFcn = createCallbackFcn(app,
@OpenImageButtonPushed, true);
        app.ConservativeButton.ButtonPushedFcn = createCallbackFcn(app,
@ConservativeButtonPushed, true);
        app.MedianButton.ButtonPushedFcn = createCallbackFcn(app,
@MedianButtonPushed, true);
        app.MeanButton.ButtonPushedFcn = createCallbackFcn(app,
@MeanButtonPushed, true);
        app.SharpenButton.ButtonPushedFcn = createCallbackFcn(app,
@SharpenButtonPushed, true);
        app.ResetButton.ButtonPushedFcn = createCallbackFcn(app,
@ResetButtonPushed, true);
        app.SaveButton.ButtonPushedFcn = createCallbackFcn(app,
@SaveButtonPushed, true);
        app.BrightnessSlider.ValueChangedFcn = createCallbackFcn(app,
@BrightnessSliderValueChanged, true);
        app.ContrastSlider.ValueChangedFcn = createCallbackFcn(app,
@ContrastSliderValueChanged, true);

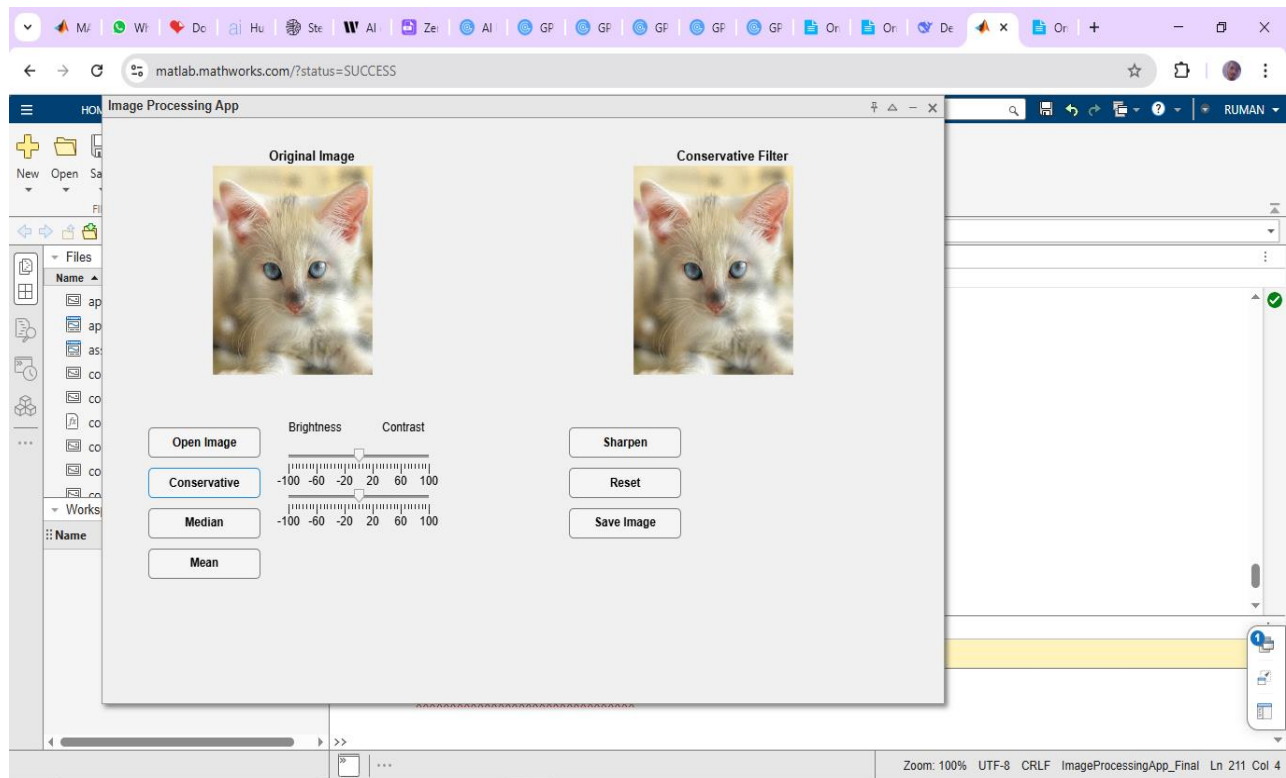
        registerApp(app, app.UIFigure);
        runStartupFcn(app, @startupFcn);
    end
end

```

```

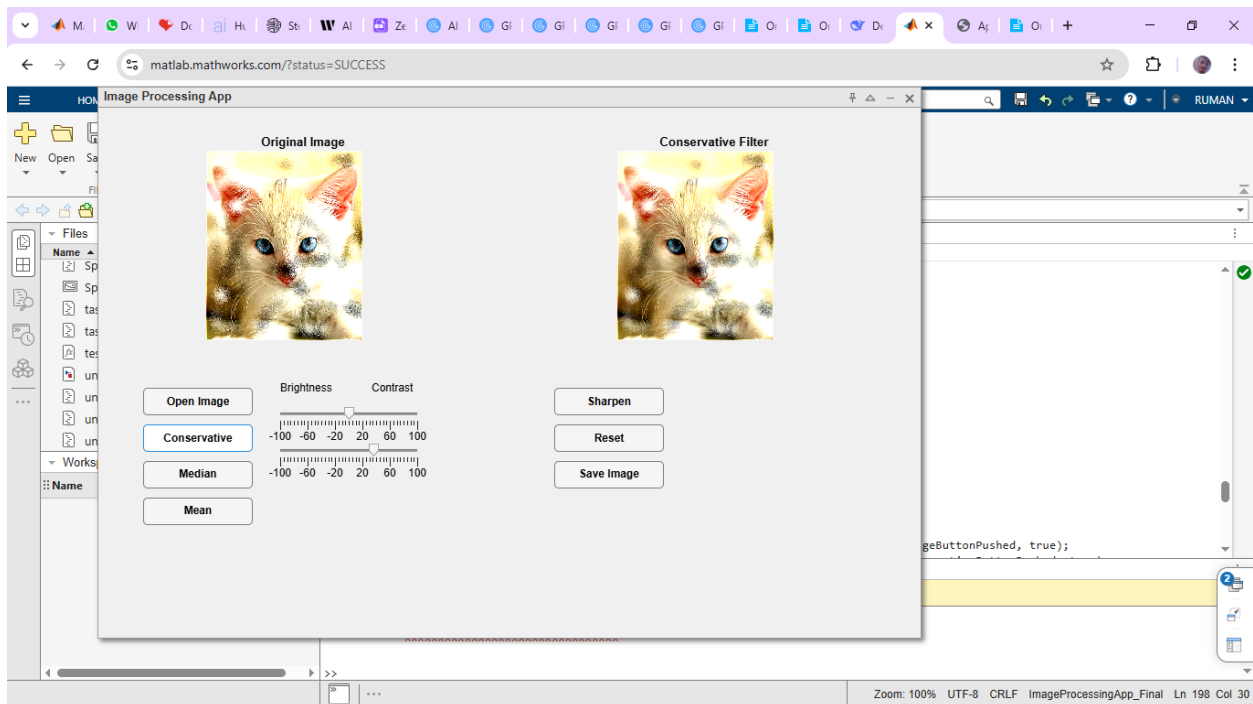
function delete(app)
    delete(app.UIFigure);
end
end
end

```



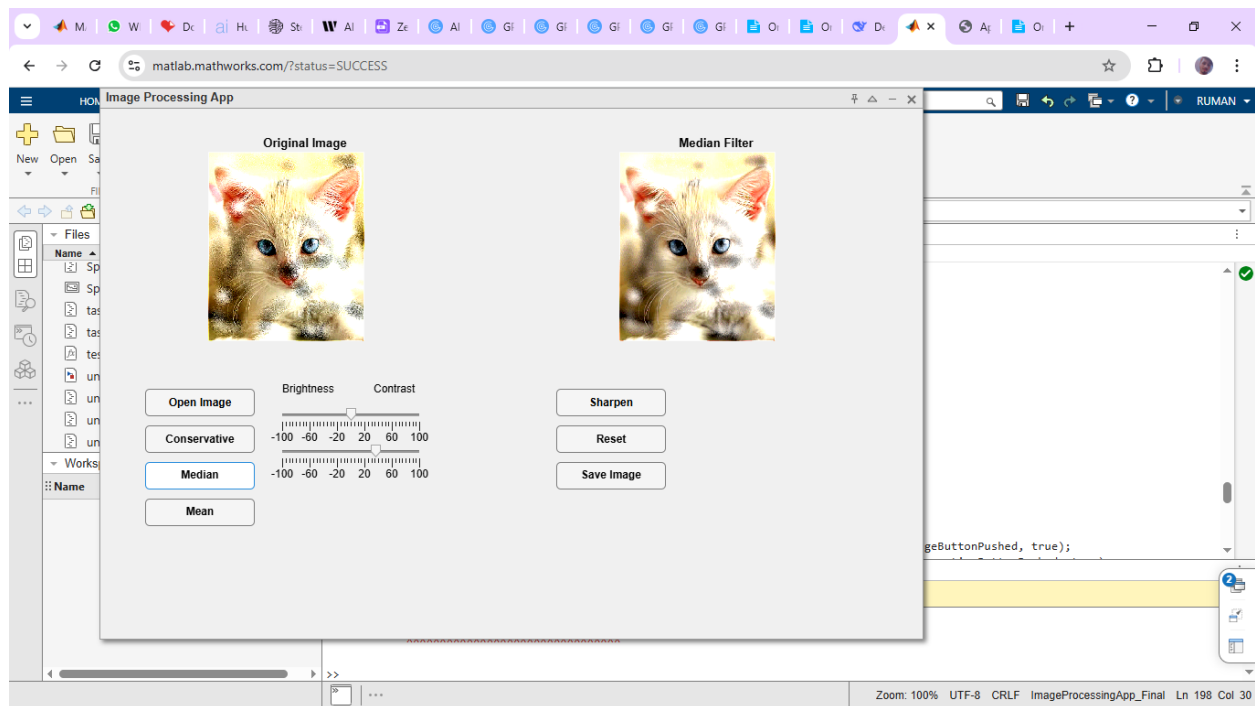
7. Image processing app applying conservative filter

1. Original image (Noisy_image.jpg) is added
2. Contrast is increased to 45
3. Sharpen is added
4. Conservative filter result is shown in the figure



8. Image processing app applying Median filter

1. Original image (Noisy_image.jpg) is added
2. Contrast is increased to 45
3. Sharpen is added
4. Median filter result is shown in the figure



9. Image processing app applying Mean filter

1. Original image (Noisy_image.jpg) is added
2. Contrast is increased to 45
3. Sharpen is added
4. Median filter result is shown in the figure

